

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer-Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Both processes should acquire ..

Semaphore empty = 5
Semaphore full = 0
Semaphore mutex = 1

Producer:
wait(mutex)
wait(empty)
insert(item)
signal(full)
signal(mutex)

Consumer:
wait(mutex)
wait(full)
remove(item)
signal(empty)
signal(mutex)

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

Q7. Thread Performance Debugging

(10 Marks)

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.

(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.

processes should acquire resources in the same order.

1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.
3. Suggest two recovery techniques and justify which one minimises system disruption.

Q10. Monitor-Based Synchronization Debugging

(10 Marks)

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
    boolean busy = false;

    procedure Print()
    {
        if(busy)
            wait();

        busy = true;

        // Printing

        busy = false;
    }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Overall Total (100)						

Name: R. Indhu

Register No: 192524332

Course: Operating System

Course Code: CSA 0404

1. Deadlock Debugging

1. Identify the exact condition that causes deadlock:

A deadlock occurs because both processes acquire the resources in a different order.

• Process P1:

- locks R1
- Waits for 100 ms
- Tries to lock R2

• Process P2:

- locks R2
- Waits for 100ms
- Tries to lock R1.

If P1 locks R1 and P2 locks R2 at the same time, then:

- P1 waits R2, which is held by P2.
- P2 waits for R1, which is held by P1.

Conditions:

- Mutual Exclusion
- Hold and Wait
- No preemption
- Circular Wait

2. Suggest an optimisation to eliminate the deadlock without changing process logic Optimisation: Use a fixed resource ordering:

Both processes should acquire resources in the same order:

Process P1:

lock(R1)

lock(R2)

Critical Section

Unlock(R2)

Unlock(R1)

Process P2:

lock(R1)

lock(R2)

Critical Section

unlock(R2)

unlock(R1)

Since both processes request R1 first and R2 second, the circular wait condition is removed, preventing deadlock.

2. CPU Scheduling Optimisation:

Problem Analysis:

The given CPU scheduling algorithm results in high waiting time & low CPU utilisation. Long processes occupy the CPU for a longer duration, causing short processes to wait unnecessarily.

This increases the average waiting time, turnaround time, and reduces overall system performance.

Issues in the Existing Algorithm:

1. High average waiting time.
2. Low CPU utilisation due to inefficient scheduling.
3. Increased turnaround time.
4. Poor response time for short processes.
5. Reduced system throughput.

Optimisation Technique:

Use the Shortest Job First (SJF) scheduling algorithm or shortest Remaining Time First (SRTF) for preemptive scheduling.

Working of SJF

- The scheduler selects the process with the smallest CPU burst time.
- Shorter processes finish earlier.
- Waiting time and turnaround time are reduced.
- CPU remains active with minimal idle time.

Advantages:

- Reduces average waiting time.
- Improves turnaround time.
- Increases CPU utilisation.

- Improves system throughput.
- Provides faster response for short processes.
- Enhances overall scheduling efficiency.

Conclusion:

The CPU scheduling algorithm can be optimised by replacing A with Shortest Job first or Shortest Remaining Time first (SRTF). These algorithms minimise waiting time, improve turnaround time, maximise CPU utilization and maximise and increase overall system performance, making the operating system more efficient.

3. Memory Bottleneck Debugging:

1. Identify the problem in memory management.

The memory management system suffers from memory fragmentation.

- When processes are allocated and deallocated, free memory is allocated.

Memory usage:

* Code = 5 MB

* Heap = 150 MB

* Stack = $25 \times 1 \text{ MB} = 25 \text{ MB}$

* Total memory Used = 180 MB

ii) Optimisation 1 - Reduce Thread Memory Usage:

* Use a thread pool instead of creating many threads.

* Reduce stack size

* Reuse thread to Reuse memory allocation.

(iii) Optimization 2 - Improve Memory Management:

* Improve data location so frequently used data remains in RAM.

* Use efficient page replacement algorithms

* Reduce unnecessary memory access.

Starvation:

i) Root Cause of starvation.

Starvation occurs when a process waits indefinitely because higher-priority process continuously receive CPU time.

• Q_1 (Highest priority) (Time Quantum = 5ms)

• Q_2 (Time Quantum = 10ms)

• Q_3 (Batch process (FCFS))

Process P_5 is in Q_3 . Since the scheduler always executes higher-priority queues (Q_1 and Q_2) first, P_5 keeps waiting whenever new interactive or system process arrive.

(ii) Effects of Starvation:

- * P_5 waits indefinitely without execution
- * CPU time is continuously allocated to higher-priority process.
- * System fairness decreases.
- * Reduces overall system efficiency.

5. Content - Switch Overhead Debugging:

- * The server performs 20,000 content switches per second, causing high CPU overhead.
- * The main reason is very small time quantum (1ms) used by the preemptive scheduler.
- * Since the average CPU burst time of each thread is 2ms, most threads are interrupted before completing their execution.

(i) Optimization 1 - Increase Time Quantum:

- * Increase the scheduler's time quantum from

1ms to a larger value (such as 4-5 ms)

* This allow threads to complete more of their execution before being preempted.

(ii) Optimisation 2 - Reduce Thread Switching:

* Use thread pooling instead of creating or switching b/w many threads.

* Reduce unnecessary I/O operations by using buffering.

* Schedule CPU-bound and I/O bound tasks efficiently to minimize frequent switching.

6. Producer - Consumers Debugging (semaphore error):

i) Identify the synchronization error causing the hang:

The synchronization error is the incorrect order of semaphore. In both the producer and consumer the meter semaphore is locked before checking the empty / full semaphore

- Producer: wait (mutex) $\rightarrow$ wait (empty)
- Consumer: wait (mutex) $\rightarrow$ wait (full)

$\Rightarrow$ Correct Semaphore Sequence:

Correct producer:

wait (empty)

wait (mutex)

insert (item)

signal (mutex)

signal (full)

Correct Consumer:

wait (full)

wait (mutex)

remove (item)

signal (mutex)

signal (empty)

f. Banker's Algorithm Debugging:

* The Banker's Algorithm reports an unsafe state because the available resource are not sufficient to satisfy the remaining resource needs of all process in a safe sequence.

Effects include:

* No safe sequence exists.

* Processes remain waiting for resources.

* Deadlock may occur in the future.

* System reliability decreases.

Optimisation to make the system safe:

- * Release resource held by completed or idel processes.

- * Allocate resources only if the request keeps the system in a safe state.

8. Monitor Synchronization Debugging:

(i) Synchronization Issue:

- The synchronization issue is the incorrect use of condition variable in the monitor. A thread calls signal () before.

- As a result, the signal is lost, and the waiting thread remains blocked forever.

(ii) Why the monitor hangs:

- The monitor hangs because:

- A thread enters the monitor & sends a signal ()

- At that moment, no other thread is waiting, so signal is lost.

- Later, another thread executes wait() and goes to sleep.

Example:

```
while (condition == false)
```

```
wait();
```

```
perform operation;
```

```
signal();
```

9. Paging Debugging:

(i) The cause of high page fault rate:

- * The high page fault rate occurs because the system has too few frames allocated to the process.

- * As the process accesses many different pages, the operating system.

(ii) The impact on System performance:

High page faults have several negative effects.

- * CPU utilization decreases because the CPU waits for disk operation.

- * Execution time increases

- * Disk I/O increases significantly.

Optimisations:

→ Allocate more (people) page frames to the process

→ * Use algorithm such as least Recently used (LRU) or optimal page replacement.

10. Deadlock Recovery Debugging:

(i) Issue in the recovery mechanism:

* The deadlock recovery mechanism always Terminates the highest - priority process.

* Terminating them causes loss of important work, reduce system reliability.

(ii) The impact on system performance:

High page faults have several negative effects

* CPU utilization decreases because the CPU waits for disk operation.

* Execution time increases

* Disk I/O increases significantly.

* System throughput decreases.

Optimization:

* Select the lowest - priority process or the process with the least recovery cost as victim

* Use resource preemption or Banker's Algorithm to avoid deadlocks.